

Sistem Brankas Digital untuk Pengamanan Data Menggunakan Kriptografi AES-256 Berbasis Web

Muhammad Afif Dhiyaulhaq¹⁾, Muhammad Arafa Yahfahzka²⁾, Rafly Ridho' Sukardi³⁾, Heri Wijayanto⁴⁾

¹⁾ Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: afifdhiyaulhaq01@gmail.com, fazarafa7@gmail.com, fld02310134@unram.ac.id

Abstract

Perkembangan teknologi digital telah meningkatkan kebutuhan akan sistem penyimpanan data yang aman dan mampu melindungi privasi pengguna dari berbagai ancaman keamanan siber. Meskipun layanan penyimpanan berbasis cloud semakin banyak digunakan, sebagian besar layanan tersebut masih mengandalkan model keamanan yang memberikan akses terhadap data pengguna kepada penyedia layanan. Penelitian ini mengembangkan CIPHERVAULT, sebuah sistem brankas digital berbasis web yang menerapkan konsep Zero-Knowledge Architecture untuk memastikan bahwa penyedia layanan tidak memiliki akses terhadap data pengguna. Sistem menggunakan algoritma AES-256-GCM sebagai mekanisme enkripsi utama dan PBKDF2-HMAC-SHA256 dengan 310.000 iterasi untuk menghasilkan kunci kriptografi yang kuat dari kata sandi pengguna. Seluruh proses enkripsi dan dekripsi dilakukan di sisi klien menggunakan Web Crypto API, sehingga server hanya menerima dan menyimpan data dalam bentuk ciphertext. Selain menyediakan fitur penyimpanan file terenkripsi dan secure notes, sistem juga dilengkapi dengan manajemen kata sandi dan activity logging untuk meningkatkan keamanan serta pemantauan aktivitas pengguna. Hasil implementasi menunjukkan bahwa CIPHERVAULT mampu menjaga kerahasiaan, integritas, dan privasi data pengguna, serta tetap memberikan perlindungan terhadap data meskipun terjadi kebocoran database, serangan man-in-the-middle, maupun kompromi pada server penyimpanan.

Keywords: Zero-Knowledge Architecture, AES-256-GCM, PBKDF2, Web Crypto API, keamanan data, client-side encryption.

Abstract

The rapid growth of digital technology has increased the demand for secure data storage systems capable of protecting user privacy against various cybersecurity threats. Although cloud-based storage services are widely used, most of them still rely on security models that allow service providers to access user data. This study develops CIPHERVAULT, a web-based digital vault system that implements a Zero-Knowledge Architecture approach to ensure that service providers have no access to user data. The system utilizes AES-256-GCM as the primary encryption algorithm and PBKDF2-HMAC-SHA256 with 310,000 iterations to derive strong cryptographic keys from user passwords. All encryption and decryption processes are performed on the client side using the Web Crypto API, ensuring that the server only receives and stores ciphertext. In addition to encrypted file storage and secure notes, the system provides password management and activity logging features to enhance security and user activity monitoring. The implementation results demonstrate that CIPHERVAULT effectively preserves data confidentiality, integrity, and privacy while maintaining protection against database breaches, man-in-the-middle attacks, and server compromise incidents.

Keyword: Zero-Knowledge Architecture, AES-256-GCM, PBKDF2, Web Crypto API, data security, client-side encryption.

1. PENDAHULUAN

Perkembangan teknologi digital yang pesat membawa dampak signifikan terhadap cara penyimpanan dan pertukaran informasi sensitif. Individu maupun organisasi semakin bergantung pada layanan penyimpanan berbasis *cloud* untuk menyimpan dokumen penting, catatan pribadi, dan berkas rahasia. Namun, mayoritas layanan *cloud* konvensional menyimpan data pengguna dalam kondisi yang dapat diakses oleh penyedia layanan itu sendiri, sehingga menimbulkan risiko kebocoran data baik melalui

serangan siber maupun penyalahgunaan dari dalam (*insider threat*) [1].

Statistik global menunjukkan bahwa pelanggaran data (*data breach*) terus meningkat setiap tahunnya. Model keamanan tradisional yang mengandalkan kepercayaan kepada pihak ketiga (server/penyedia layanan) terbukti rentan. Dibutuhkan paradigma baru yang dikenal sebagai *Zero-Knowledge Architecture*, di mana penyedia layanan secara matematis tidak memiliki kemampuan untuk mengakses data pengguna meskipun mereka menginginkannya [1].

Berdasarkan permasalahan tersebut, kelompok kami mengembangkan CIPHERVAULT, sebuah

aplikasi brankas digital berbasis web yang menerapkan enkripsi sisi klien (*client-side encryption*) secara penuh. Seluruh proses kriptografi terjadi di browser pengguna sebelum data dikirimkan ke server, sehingga server hanya menyimpan data acak (*ciphertext*) yang tidak bermakna tanpa kunci dari pengguna [2].

1.1. Tujuan

Berdasarkan latar belakang yang telah diuraikan, maka Tujuan dalam penelitian ini adalah sebagai berikut:

- Mengimplementasikan sistem penyimpanan file dan catatan terenkripsi berbasis web dengan arsitektur *Zero-Knowledge* menggunakan algoritma AES-256-GCM.
- Menerapkan *Key Derivation Function* PBKDF2 dengan iterasi tinggi (310.000 iterasi) untuk memproteksi kunci enkripsi dari serangan *brute-force*.
- Membangun antarmuka pengguna yang intuitif dan aman menggunakan Web Crypto API standar browser tanpa ketergantungan library kriptografi pihak ketiga.
- Menyediakan fitur audit trail (*activity logging*) untuk pemantauan aktivitas sensitif pengguna sebagai lapisan keamanan tambahan.
- Mendemonstrasikan penerapan nyata konsep kriptografi modern dalam konteks rekayasa perangkat lunak berbasis web (Laravel + Vite/Tailwind CSS).

2. TINJAUAN PUSTAKA

2.1. AES-256-GCM (Advanced Encryption Standard – Galois/Counter Mode)

AES adalah algoritma enkripsi simetris yang diadopsi oleh NIST (National Institute of Standards and Technology) sebagai standar enkripsi federal Amerika Serikat sejak tahun 2001. Dengan panjang kunci 256-bit, AES-256 menawarkan tingkat keamanan tertinggi dalam keluarga AES dan digunakan secara luas oleh lembaga pemerintah, militer, dan industri keuangan di seluruh dunia [3][4].

Mode operasi GCM (*Galois/Counter Mode*) merupakan mode authenticated *encryption* yang memberikan dua jaminan sekaligus: kerahasiaan (*confidentiality*) melalui enkripsi CTR dan integritas data (*data integrity/authenticity*) melalui *Galois Message Authentication Code* (GMAC). Properti ini memastikan bahwa *ciphertext* yang telah dimodifikasi oleh pihak tidak berwenang akan gagal saat proses dekripsi, sehingga melindungi sistem dari serangan tampering [3][5].

Dalam implementasi CIPHERVAULT, AES-256-GCM dieksekusi melalui Web Crypto API (`window.Crypto.subtle`) yang merupakan implementasi kriptografi native dan terpercaya yang tersedia di seluruh browser modern.

2.2. PBKDF2 (Password-Based Key Derivation Function 2)

PBKDF2 adalah fungsi penurunan kunci berbasis kata sandi yang didefinisikan dalam RFC 2898 dan direkomendasikan oleh NIST SP 800-132. Fungsi ini dirancang untuk mengatasi kelemahan mendasar dari kata sandi manusia yang cenderung singkat dan mudah ditebak, dengan mengubah kata sandi tersebut menjadi kunci kriptografi yang kuat melalui proses iteratif [6].

Mekanisme PBKDF2 menerapkan fungsi pseudorandom (dalam proyek ini menggunakan HMAC-SHA-256) secara berulang sebanyak jumlah iterasi yang ditentukan terhadap kombinasi kata sandi dan salt acak. Penggunaan salt mencegah serangan precomputation (*rainbow table*), sementara jumlah iterasi yang tinggi memperlambat laju percobaan *brute-force* secara signifikan. CIPHERVAULT menggunakan 310.000 iterasi, sesuai dengan rekomendasi terbaru OWASP untuk PBKDF2-SHA256.

2.3. Initialization Vector (IV) dan Salt

Initialization Vector (IV) adalah nilai acak yang digunakan sebagai input tambahan dalam algoritma enkripsi blok mode operasi seperti GCM. IV memastikan bahwa enkripsi data yang identik dengan kunci yang sama akan menghasilkan *ciphertext* yang berbeda-beda, sehingga mencegah analisis pola (*pattern analysis*). Standar untuk AES-GCM merekomendasikan IV sepanjang 96-bit (12 byte) [7].

Salt adalah nilai acak yang digunakan dalam proses *Key Derivation* untuk memastikan keunikan kunci yang dihasilkan meskipun kata sandi yang digunakan sama. CIPHERVAULT meng-generate salt baru (256-bit/32 byte) untuk setiap operasi enkripsi menggunakan `window.Crypto.getRandomValues()` yang merupakan generator angka acak yang aman secara kriptografi (CSPRNG) [8].

2.4. Arsitektur Zero-Knowledge

Zero-Knowledge Architecture (ZKA) dalam konteks penyimpanan data merupakan konsep keamanan modern yang dirancang agar penyedia layanan tidak memiliki kemampuan untuk melihat, membaca, maupun mengakses isi data pengguna. Pada arsitektur ini, seluruh proses enkripsi dilakukan langsung di perangkat pengguna sebelum data dikirim ke server. Dengan demikian, server hanya menerima data dalam bentuk terenkripsi (*ciphertext*), sedangkan kunci dekripsi tetap berada di sisi pengguna dan tidak

pernah dikirim ataupun disimpan oleh penyedia layanan [9].

Konsep ini berbeda dengan sistem keamanan biasa yang hanya menggunakan enkripsi transport seperti HTTPS. HTTPS memang melindungi data saat proses pengiriman melalui internet, namun setelah data sampai di server, data tersebut masih dapat dibaca oleh penyedia layanan. Sementara itu, pada *Zero-Knowledge Architecture*, data tetap terenkripsi bahkan setelah disimpan di server sehingga pihak penyedia layanan tidak mengetahui isi asli data pengguna [10].

Model keamanan ini memiliki implikasi yang sangat penting terhadap perlindungan privasi dan keamanan informasi. Apabila terjadi kebocoran database, peretasan server, ataupun pencurian data, file yang tersimpan tetap tidak dapat dibaca tanpa kunci dekripsi yang dimiliki pengguna. Bahkan dalam kondisi tertentu ketika penyedia layanan diminta menyerahkan data kepada pihak lain, data yang diberikan tetap berada dalam bentuk terenkripsi dan tidak dapat dipahami tanpa akses terhadap kunci pengguna [11].

Pendekatan *Zero-Knowledge Architecture* banyak digunakan pada layanan keamanan modern seperti ProtonMail untuk email terenkripsi, Tresorit untuk penyimpanan *cloud* aman, serta Signal untuk komunikasi terenkripsi *end-to-end*. Penggunaan konsep ini menunjukkan bahwa keamanan data tidak hanya bergantung pada perlindungan server, tetapi juga pada kontrol penuh pengguna terhadap kunci enkripsi dan akses terhadap data mereka sendiri.

3. METODOLOGI PENELITIAN

Penelitian ini dilakukan melalui beberapa tahapan yang saling berkaitan dalam pengembangan sistem brankas digital berbasis web CIPHERVAULT. Kegiatan diawali dengan studi literatur untuk mempelajari konsep keamanan data, kriptografi modern, *Zero-Knowledge Architecture*, serta standar keamanan aplikasi web yang direkomendasikan oleh OWASP. Selain itu, dilakukan analisis terhadap berbagai ancaman keamanan yang umum terjadi pada sistem penyimpanan data digital, seperti kebocoran database, akses tidak sah, pencurian kredensial, dan serangan terhadap data sensitif pengguna. Hasil kajian tersebut menjadi dasar dalam merancang arsitektur sistem yang mampu memberikan perlindungan terhadap data pengguna melalui mekanisme enkripsi pada sisi klien (*client-side encryption*).

Setelah tahap analisis selesai, dilakukan perancangan sistem yang meliputi perancangan basis data, antarmuka pengguna, alur autentikasi, serta mekanisme enkripsi dan dekripsi data. Sistem dikembangkan menggunakan bahasa pemrograman PHP dengan framework Laravel 12 sebagai platform backend, SQLite sebagai basis data, serta Web Crypto API untuk mengimplementasikan fungsi kriptografi

pada sisi klien. Perancangan juga menerapkan konsep *Zero-Knowledge Architecture* sehingga server tidak memiliki akses terhadap data asli pengguna yang tersimpan dalam sistem.

Implementasi sistem diwujudkan melalui beberapa fitur utama, yaitu autentikasi pengguna, penyimpanan file terenkripsi, secure notes, serta activity logging. Ketika pengguna mengunggah file atau menyimpan catatan rahasia, data terlebih dahulu diproses pada browser pengguna. *Password* yang dimasukkan pengguna digunakan sebagai dasar pembentukan kunci kriptografi melalui algoritma PBKDF2-HMAC-SHA256 dengan bantuan salt yang dihasilkan secara acak. Kunci yang dihasilkan kemudian digunakan untuk melakukan proses enkripsi menggunakan algoritma AES-256-GCM sebelum data dikirim ke server.

Hasil enkripsi berupa *ciphertext* beserta parameter kriptografi seperti Initialization Vector (IV) dan salt disimpan pada database, sedangkan data asli (*plaintext*) tidak pernah dikirim maupun disimpan pada server. Pada proses dekripsi, pengguna diwajibkan memasukkan *Password* yang sama dengan *Password* yang digunakan saat proses enkripsi. Sistem kemudian membangkitkan kembali kunci kriptografi menggunakan PBKDF2-HMAC-SHA256 dan melakukan proses dekripsi terhadap *ciphertext* yang tersimpan. Apabila *Password* yang dimasukkan tidak sesuai, proses dekripsi akan gagal sehingga data tidak dapat diakses.

Setelah implementasi selesai, dilakukan pengujian keamanan sistem berdasarkan beberapa kategori yang mengacu pada OWASP Top 10, yaitu SQL Injection, Broken Access Control, *Authentication Failures*, Cryptographic Failures, Security Logging and Monitoring, serta simulasi kebocoran database (*Data Breach Simulation*). Pengujian dilakukan untuk mengevaluasi kemampuan sistem dalam melindungi data pengguna dari berbagai ancaman keamanan yang umum terjadi pada aplikasi berbasis web. Hasil pengujian kemudian dianalisis untuk menilai efektivitas penerapan kriptografi AES-256-GCM, PBKDF2-HMAC-SHA256, dan *Zero-Knowledge Architecture* dalam menjaga kerahasiaan, integritas, dan keamanan data pengguna.

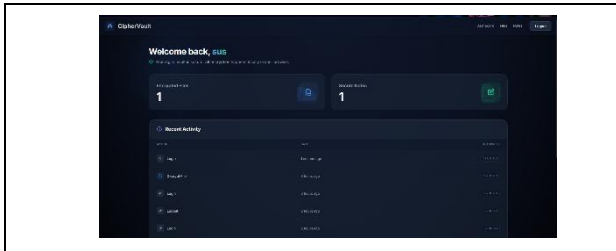
4. HASIL DAN PEMBAHASAN

4.1. Hasil Implementasi Sistem

CIPHERVAULT berhasil dikembangkan sebagai aplikasi brankas digital berbasis web yang berfungsi untuk menyimpan dan melindungi data sensitif pengguna menggunakan mekanisme kriptografi modern. Sistem dibangun menggunakan framework Laravel 12 dengan SQLite sebagai basis data dan memanfaatkan Web Crypto API untuk melakukan

proses enkripsi dan dekripsi pada sisi klien (*client-side encryption*).

Aplikasi menyediakan beberapa fitur utama, yaitu autentikasi pengguna, penyimpanan file terenkripsi, secure notes, activity logging, serta mekanisme dekripsi lokal menggunakan *Password* yang dimiliki pengguna. Seluruh data sensitif diproses terlebih dahulu pada browser pengguna sebelum dikirim dan disimpan ke server.



4.2. Implementasi Kriptografi

Mekanisme keamanan utama pada CIPHERVAULT menggunakan algoritma AES-256-GCM sebagai metode enkripsi simetris. Algoritma ini dipilih karena mampu memberikan kerahasiaan (*confidentiality*) dan integritas (*integrity*) data secara bersamaan.

Kunci enkripsi tidak disimpan secara langsung oleh sistem. Sebagai gantinya, kunci dihasilkan menggunakan algoritma PBKDF2-HMAC-SHA256 yang memanfaatkan *Password* pengguna dan nilai salt acak. Pendekatan ini meningkatkan ketahanan sistem terhadap serangan *brute force* maupun dictionary attack.

Setiap proses enkripsi menghasilkan *ciphertext*, Initialization Vector (IV), dan salt yang kemudian disimpan pada database. Data asli (*plaintext*) tidak pernah disimpan pada server.

4.3. Implementasi Basis Data

Sistem menggunakan SQLite sebagai media penyimpanan data. Berdasarkan hasil implementasi, database terdiri dari beberapa tabel utama yang mendukung proses operasional sistem, yaitu tabel *users*, *encrypted_files*, *secure_notes*, dan *activity_logs*.

Tabel *users* digunakan untuk menyimpan informasi akun pengguna, tabel *encrypted_files* digunakan untuk menyimpan metadata file terenkripsi, tabel *secure_notes* digunakan untuk menyimpan catatan rahasia pengguna, sedangkan tabel *activity_logs* digunakan untuk mencatat aktivitas yang terjadi dalam sistem.

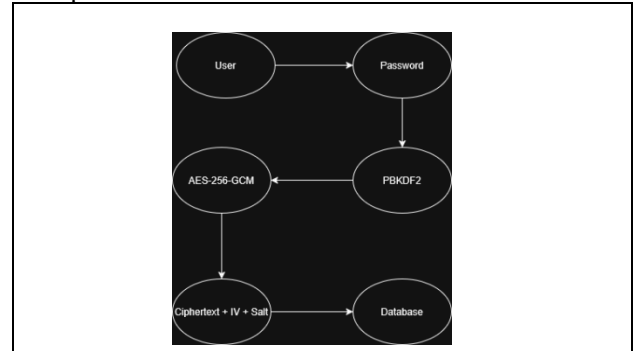
Hubungan antar tabel dirancang menggunakan *foreign Key* berupa *user_id* sehingga setiap data yang tersimpan dapat dikaitkan dengan pemiliknya masing-masing.

4.4. Implementasi Zero-Knowledge Architecture

CIPHERVAULT menerapkan konsep *Zero-Knowledge Architecture* yang bertujuan memastikan bahwa server tidak memiliki akses terhadap isi data pengguna. Pada arsitektur ini, seluruh proses enkripsi dilakukan pada browser menggunakan Web Crypto API sebelum data dikirim ke server.

Ketika pengguna mengunggah file atau menyimpan *secure notes*, sistem terlebih dahulu menghasilkan kunci enkripsi dari *Password* pengguna menggunakan PBKDF2. Data kemudian dienkripsi menggunakan AES-256-GCM dan menghasilkan *ciphertext*. Hanya *ciphertext* beserta parameter kriptografi yang dikirim ke server untuk disimpan.

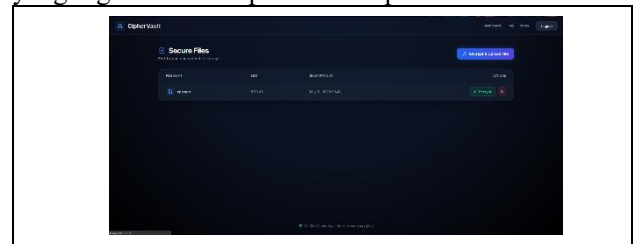
Dengan pendekatan ini, administrator sistem maupun pihak yang memperoleh akses ke database tidak dapat membaca isi data pengguna tanpa mengetahui *Password* yang digunakan dalam proses enkripsi.



4.5. Implementasi Penyimpanan File Aman

Fitur *Secure Files* memungkinkan pengguna mengunggah berbagai jenis file untuk disimpan secara aman. Sebelum file dikirim ke server, isi file akan dienkripsi menggunakan AES-256-GCM pada sisi klien. Setelah proses enkripsi berhasil, sistem hanya menyimpan *ciphertext* dan metadata yang diperlukan untuk proses dekripsi.

Berdasarkan hasil implementasi, file yang tersimpan pada database tidak dapat dibaca secara langsung karena telah mengalami proses enkripsi. Untuk mengakses kembali file tersebut, pengguna harus memasukkan *Password* yang sama dengan *Password* yang digunakan saat proses enkripsi dilakukan.

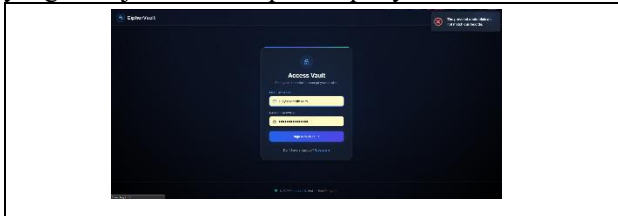


4.6. Pengujian SQL Injection (A05 – Injection)

Pengujian SQL Injection dilakukan pada fitur autentikasi dengan memasukkan input yang tidak sesuai pada form login untuk menguji apakah sistem rentan terhadap manipulasi query database. Pengujian dilakukan dengan mencoba berbagai kombinasi input yang menyerupai payload SQL Injection pada field email maupun *Password*.

Hasil pengujian menunjukkan bahwa sistem menolak seluruh percobaan login yang menggunakan kredensial tidak valid. Sistem menampilkan pesan kesalahan dan tidak memberikan akses ke dalam aplikasi. Hal ini menunjukkan bahwa input pengguna tidak dieksekusi sebagai perintah SQL dan telah diproses dengan aman oleh mekanisme autentikasi Laravel.

Framework Laravel yang digunakan pada sistem menerapkan Eloquent ORM dan parameter binding sehingga mampu mencegah serangan SQL Injection yang bertujuan memanipulasi query database.



4.7. Pengujian Authentication Failures (A07 – Identification and Authentication Failures)

Pengujian autentikasi dilakukan dengan mencoba masuk ke sistem menggunakan kombinasi email dan *Password* yang tidak valid. Selain itu dilakukan juga pemeriksaan terhadap penyimpanan *Password* pada database.

Hasil pengujian menunjukkan bahwa sistem menolak seluruh percobaan login yang menggunakan kredensial tidak valid. Ketika email atau *Password* yang dimasukkan tidak sesuai dengan data yang tersimpan pada database, sistem menampilkan pesan kesalahan dan pengguna tidak dapat mengakses dashboard aplikasi.

Selain itu, hasil pemeriksaan database menunjukkan bahwa *Password* pengguna tidak disimpan dalam bentuk plaintext melainkan menggunakan algoritma hashing bcrypt yang ditandai dengan format hash "\$2y\$12\$". Dengan demikian, meskipun database berhasil diakses oleh pihak yang tidak berwenang, *Password* asli pengguna tidak dapat diketahui secara langsung.



4.8. Pengujian Cryptographic Failures (A02/A04 – Cryptographic Failures)

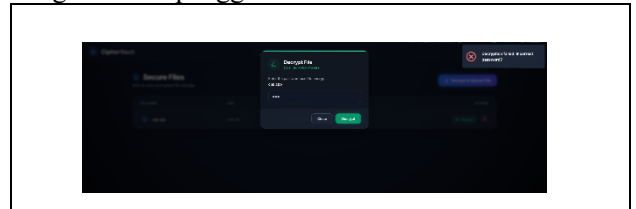
Pengujian kriptografi dilakukan untuk memastikan bahwa sistem menerapkan mekanisme enkripsi dengan benar serta tidak menyimpan data sensitif dalam bentuk plaintext.

Pengujian dilakukan dengan mengunggah file, melakukan proses enkripsi, kemudian mencoba melakukan dekripsi menggunakan *Password* yang salah. Hasil pengujian menunjukkan bahwa sistem gagal melakukan dekripsi ketika *Password* yang dimasukkan tidak sesuai dengan *Password* yang digunakan saat proses enkripsi.

Pesan kesalahan "Decryption failed. Incorrect Password?" berhasil ditampilkan oleh sistem yang menunjukkan bahwa kunci dekripsi tidak dapat direkonstruksi tanpa *Password* yang benar.

Selain itu, hasil pemeriksaan database menunjukkan bahwa file tersimpan dalam bentuk *ciphertext* yang disertai dengan parameter kriptografi berupa Initialization Vector (IV) dan salt.

Implementasi AES-256-GCM yang digunakan pada sistem mampu menjaga kerahasiaan serta integritas data pengguna.



4.9. Pengujian Data Breach Simulation

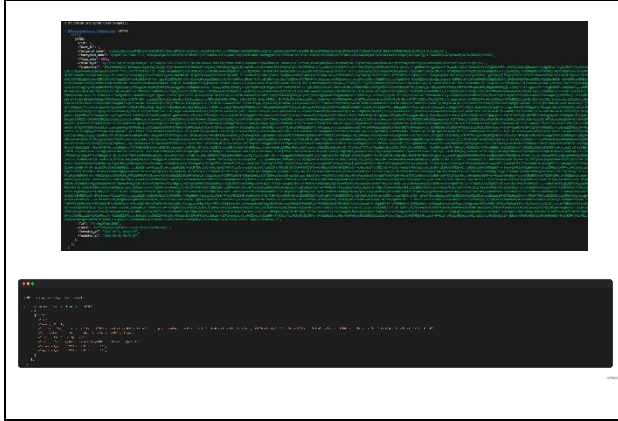
Pengujian Data Breach Simulation dilakukan dengan mensimulasikan kondisi ketika database berhasil diperoleh oleh pihak yang tidak berwenang. Tujuan pengujian ini adalah untuk mengetahui apakah data sensitif masih dapat dibaca setelah database berhasil dicuri.

Hasil pengujian dilakukan dengan memeriksa isi tabel `users`, `encrypted_files`, dan `secure_notes` menggunakan Laravel Tinker. Dari hasil pemeriksaan ditemukan bahwa data sensitif tidak tersimpan dalam bentuk plaintext. Informasi yang tersimpan pada database berupa *ciphertext*, Initialization Vector (IV), salt, dan hash *Password*.

Data yang berhasil diperoleh dari database tidak dapat langsung dibaca maupun digunakan tanpa

mengetahui *Password* pengguna yang digunakan untuk menghasilkan kunci enkripsi.

Hal ini menunjukkan bahwa konsep *Zero-Knowledge Architecture* yang diterapkan pada sistem mampu memberikan perlindungan tambahan terhadap kebocoran data apabila terjadi kompromi server maupun database.



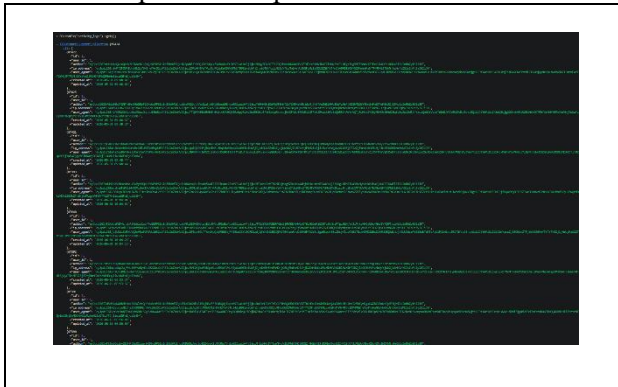
4.10. Pengujian Security Logging and Monitoring (A09 – Security Logging and Monitoring Failures)

Pengujian Security Logging dilakukan untuk memastikan bahwa aktivitas penting pengguna tercatat dengan baik oleh sistem. Pengujian dilakukan dengan mengamati isi tabel *activity_logs* setelah pengguna melakukan berbagai aktivitas seperti login, pengelolaan file, dan penggunaan fitur lainnya.

Hasil pengujian menunjukkan bahwa sistem berhasil menyimpan log aktivitas pengguna ke dalam database. Informasi yang dicatat meliputi identitas pengguna, alamat IP, user agent, waktu aktivitas, serta jenis aktivitas yang dilakukan.

Keberadaan log aktivitas ini memungkinkan administrator melakukan proses monitoring, audit keamanan, serta investigasi apabila terjadi insiden keamanan pada sistem.

Dengan adanya mekanisme logging, sistem memiliki kemampuan yang lebih baik dalam mendeteksi aktivitas mencurigakan dan meningkatkan keamanan operasional aplikasi.



5. KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi CipherVault berhasil dikembangkan sebagai sistem brankas digital berbasis web yang mampu melindungi data sensitif pengguna menggunakan algoritma kriptografi AES-256-GCM dan mekanisme derivasi kunci PBKDF2-HMAC-SHA256. Sistem menerapkan konsep *Zero-Knowledge Architecture*, di mana proses enkripsi dan dekripsi dilakukan pada sisi klien sehingga server hanya menyimpan data dalam bentuk *ciphertext* tanpa memiliki akses terhadap isi data pengguna.

Hasil pengujian keamanan berdasarkan beberapa kategori OWASP Top 10 menunjukkan bahwa sistem mampu mencegah serangan SQL Injection, menerapkan kontrol akses yang baik antar pengguna, menjaga keamanan proses autentikasi, serta melindungi data melalui implementasi kriptografi yang tepat. Simulasi kebocoran database juga menunjukkan bahwa data yang tersimpan tetap tidak dapat dibaca tanpa *Password* pengguna karena seluruh informasi sensitif disimpan dalam bentuk terenkripsi. Selain itu, fitur activity logging berhasil mencatat aktivitas pengguna sehingga mendukung proses monitoring dan audit keamanan.

Dengan demikian, CipherVault berhasil memenuhi tujuan penelitian yaitu menyediakan sistem penyimpanan data yang aman, menjaga kerahasiaan data pengguna, serta memberikan perlindungan tambahan terhadap berbagai ancaman keamanan yang umum terjadi pada aplikasi berbasis web.

5.2. Saran

Meskipun sistem telah berhasil diimplementasikan dan memenuhi tujuan yang ditetapkan, masih terdapat beberapa pengembangan yang dapat dilakukan pada penelitian selanjutnya. Pengembangan tersebut antara lain penambahan fitur multi-factor *Authentication* (MFA) untuk meningkatkan keamanan proses login, implementasi berbagi file terenkripsi (secure file sharing) antar pengguna, serta dukungan penyimpanan berbasis *cloud storage* yang tetap mempertahankan konsep *Zero-Knowledge Architecture*.

Selain itu, pengujian keamanan dapat diperluas dengan menggunakan alat profesional seperti OWASP ZAP, Burp Suite, maupun melakukan penetration testing yang lebih mendalam terhadap seluruh fitur sistem. Pengembangan pada aspek monitoring juga dapat dilakukan dengan menambahkan sistem notifikasi keamanan secara real-time apabila terdeteksi aktivitas yang mencurigakan.

Dengan pengembangan tersebut, diharapkan CipherVault dapat menjadi solusi penyimpanan data

yang lebih aman, andal, dan siap digunakan dalam lingkungan yang lebih luas.

DAFTAR PUSTAKA

- [1] F. Baso dan N. A. L., "Implementasi Teknik Kriptografi dengan Metode AES 256 untuk Keamanan File," *Information Technology Education Journal*, vol. 3, no. 3, pp. 84–87, 2024, doi: 10.59562/intec.v3i3.5525.
- [2] F. Aditya, "Implementasi Metode Kriptografi Advanced Encryption Standard 256 Bit Berbasis Web dan Mobile Pada Pengamanan Dokumen Notaris," *Technical Interaction Journal*, 2025.
- [3] A. S. Putra dan M. R. Hidayat, "Perancangan dan Implementasi Sistem Enkripsi Data Sensitif Menggunakan AES-256-CBC pada Aplikasi Berbasis Web Sederhana," *JUIKTI*, 2026.
- [4] Z. H. A. Putri dan R. D. A. Pradana, "Pengamanan Data Akademik Berbasis Web dengan Enkripsi AES-256 (Studi Kasus pada Pendataan Digital SMA XYZ)," *JNATIA*, 2025.
- [5] R. A. Suryani dan T. K. D. Putra, "Analisis Penggunaan Kriptografi Metode AES 256 Bit pada Pengamanan File dengan Berbagai Format," *Digitech*, 2025.
- [6] M. I. Rahman dan A. Fadli, "SISTEM PENGAMANAN DOKUMEN MENGGUNAKAN ALGORITMA KRIPTOGRAFI ADVANCED ENCRYPTION STANDARD (AES-256)," *JINTEKS*, 2024.
- [7] N. A. L. dan F. Baso, "Integrasi Algoritma SHA-256 dan AES untuk Pengamanan Data pada Aplikasi Berbasis Web," *Juktisi*, 2026.
- [8] A. K. Sari dan B. S. Putra, "Implementasi Algoritma AES-256 Dalam Perancangan Aplikasi Pengamanan Dokumen Digital Perusahaan Berbasis Android," *INTECOM*, 2023.
- [9] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," *NIST Special Publication 800-38A*, Dec. 2001.
- [10] E. B. Barker dan J. V. Barker, "Recommendation for Key Management — Part 1: General," *NIST Spec. Pub. 800-57 Part 1 Rev. 5*, Jan. 2020.
- [11] Mozilla Developer Network, "Web Cryptography API," *MDN Web Docs*, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
- [12] National Institute of Standards and Technology, "Specification for the Advanced Encryption Standard (AES)," *FIPS PUB 197*, Nov. 2001.
- [13] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Pearson, 2019.